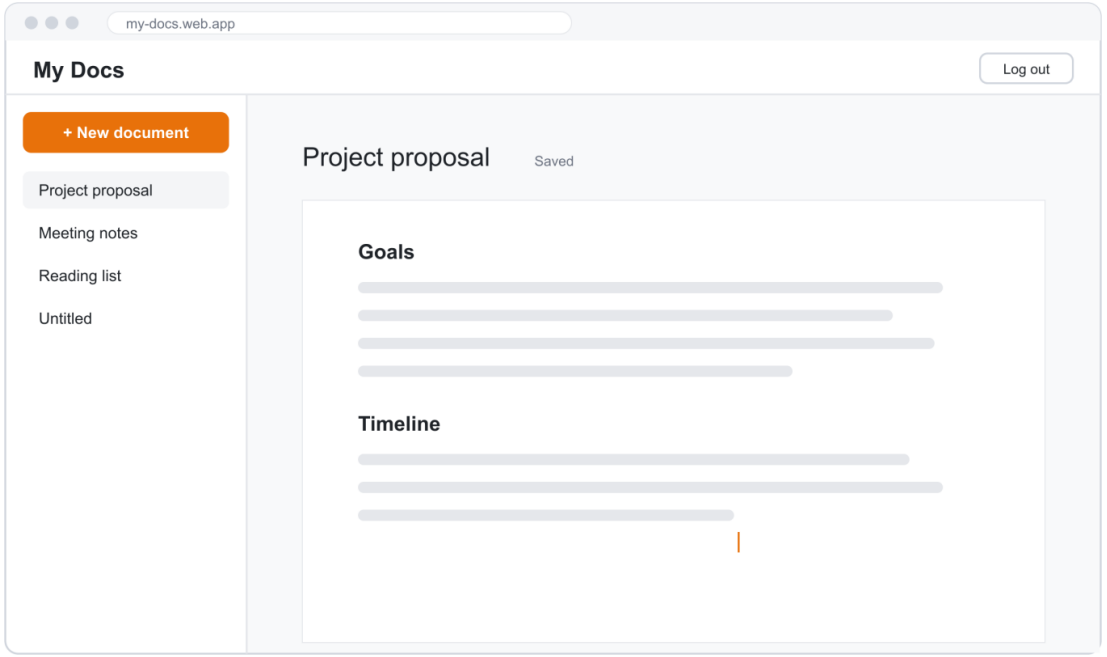
THE SIMPLE WAY

Build full-stack web apps with HTML, CSS, JS & Firebase

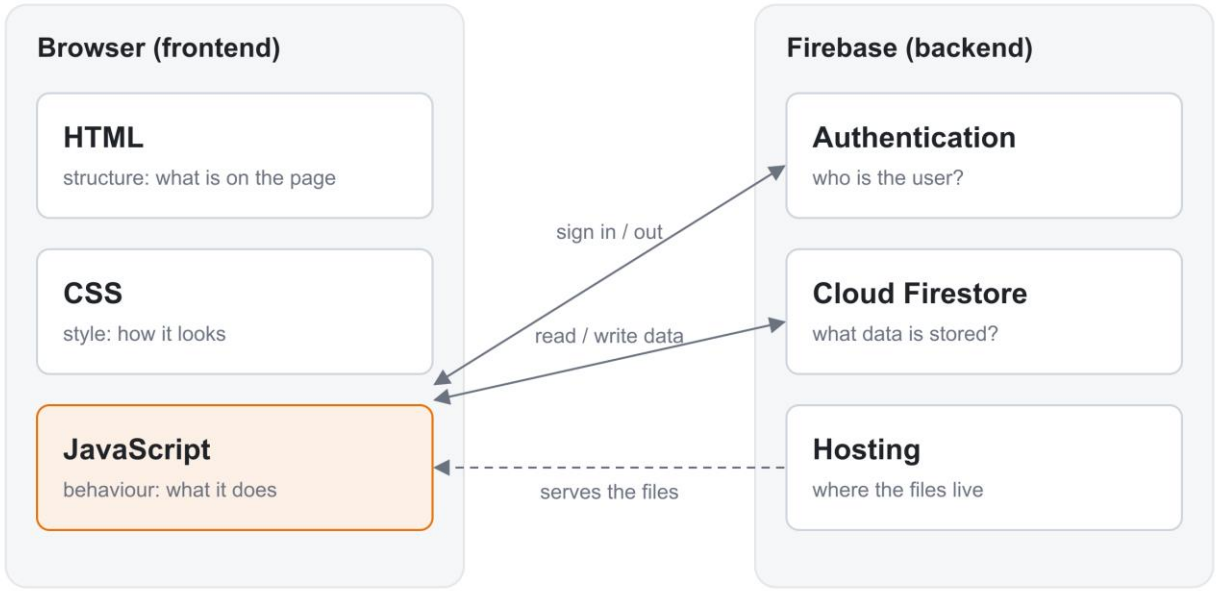
Three files, no installs, no build tools. Firebase Auth for users, Firestore for data.



What you will build: a minimal documents app

THE BIG PICTURE

Four layers, each with one job



The browser runs your three files. Firebase is the backend, so you don't write or run a server.

HTML: what is on the page
Headings, inputs, buttons, lists.

CSS: how it looks
Colours, spacing, layout.

JavaScript: what it does
Reacts to clicks, talks to Firebase, updates the page.

Firebase: the backend
Auth remembers who is logged in. Firestore saves the data.

ROADMAP

Six steps from empty folder to live app

1 **Set up**
Firebase project + three empty files

2 **HTML**
Write the page's elements

3 **CSS**
Make it look good

4 **DOM**
Make it react with JavaScript

5 **Firebase**
Log users in, save their data

6 **Ship**
Protect the data, put it online

The whole app is 3 files: `index.html`, `style.css` and `app.js`. Steps 2–4 work offline. Step 5 connects to the cloud.

STEP 1 · SET UP

Create the project and three files

In the Firebase console

1. console.firebase.google.com → Add project
2. Click the web icon </> to add a web app, then copy the config it shows you
3. Authentication → Sign-in method → turn on Email/Password
4. Firestore Database → Create database

Nothing to install for Firebase: app.js imports it from Google's CDN (step 5).

On your computer

```
folder

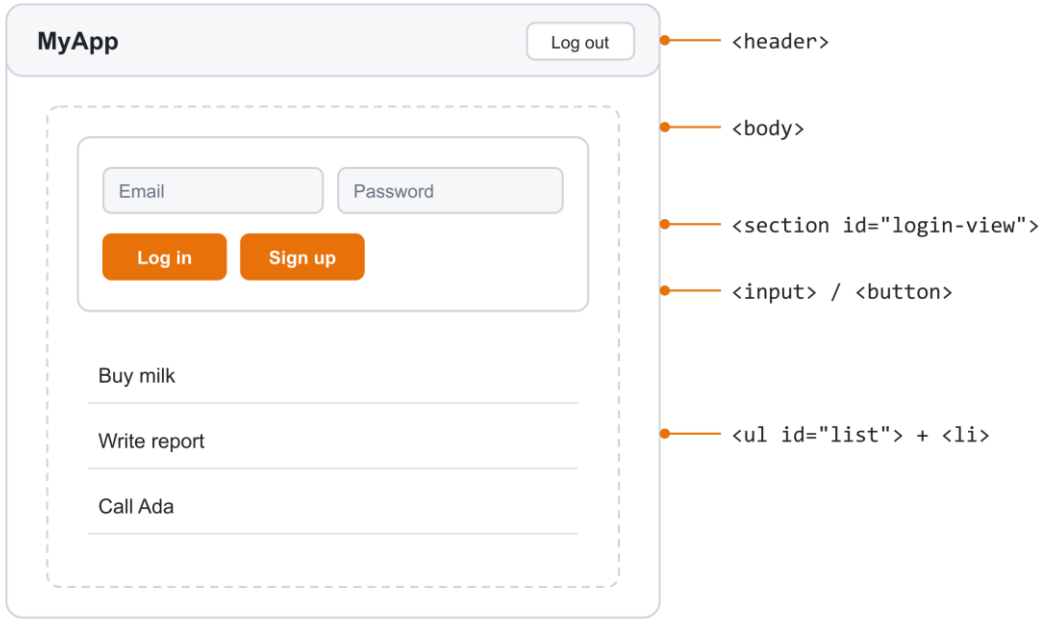
my-app/
├─ index.html   what is on the page
├─ style.css    how it looks
└─ app.js       what it does + Firebase
```

Run it locally: open the folder in VS Code and click "Go Live" (Live Server extension). Opening index.html by double-click won't work, because JS modules need a web server.

HTML is the structure: boxes inside boxes

index.html

```
<!doctype html>
<html>
<head>                    <!-- setup the user doesn't see -->
  <title>My App</title>
  <link rel="stylesheet" href="style.css">
  <script type="module" src="app.js"></script>
</head>
<body>                    <!-- what the user sees -->
  <header>
    <h1>My App</h1>
  </header>
  <section id="login-view">
    <input id="email" placeholder="Email">
    <button id="login">Log in</button>
  </section>
  <ul id="list"></ul>
</body>
</html>
```



- A tag opens and closes: <h1>Title</h1>. Tags inside tags make the structure.
- <head> links the CSS and JS files. <body> is the visible page.
- Give an element an id so JavaScript can find it.

STEP 3 · CSS

CSS picks elements, then sets how they look

style.css

```
/* selector { property: value; } */

body   { font-family: system-ui; }   /* a tag   */
.card  { padding: 24px; }            /* a class */
#error { color: crimson; }           /* an id   */
header { display: flex; }            /* layout  */
```



1 Link it
<link rel="stylesheet" href="style.css"> in the <head>

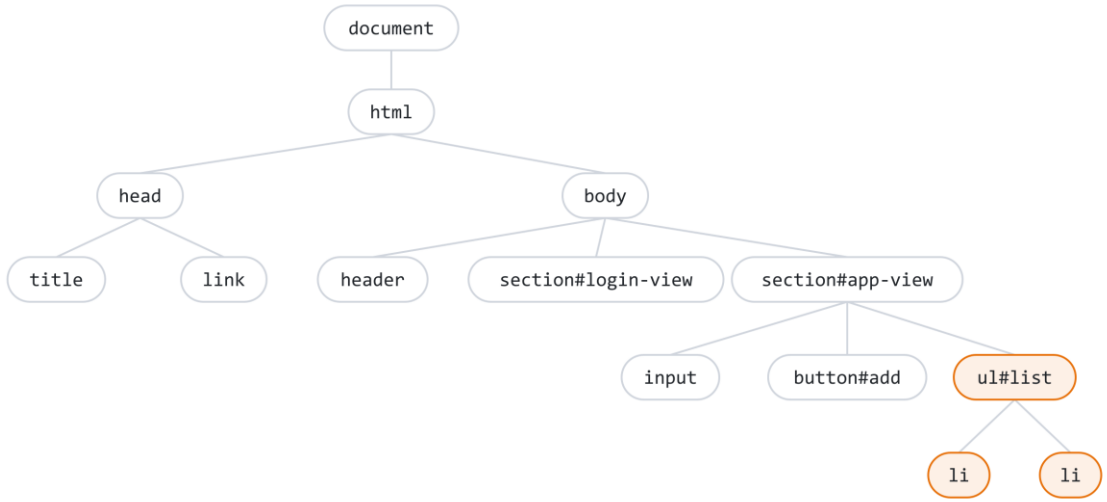
2 Select
A selector picks elements: by tag, .class or #id

3 Style
Each property: value sets one thing, like color or padding

Every element is a box
padding = space inside the border, margin = space outside it.

When two rules clash
#id beats .class beats tag. If they tie, the rule written last wins.

JavaScript changes the page through the DOM



The browser turns your HTML into a tree of objects called the DOM. JavaScript can find any of them, react to clicks on them, and add or change them. The page updates instantly.

app.js

```
// 1. Find an element by its id
const list = document.getElementById("list");

// 2. Run code when it's clicked
document.getElementById("add").addEventListener("click", () => {

  // 3. Make a new element
  const li = document.createElement("li");
  li.textContent = "New item";

  // 4. Put it on the page
  list.append(li);
});
```

Four moves do almost everything: find, listen, create, update.

STEP 5 · CONNECT FIREBASE

Load Firebase with one import line, no installs

1. **type="module"**
   This attribute on the script tag lets app.js use import.

2. **Import from the CDN**
   Paste Google's URL into the import. The browser downloads it.

3. **Paste your config**
   From the console (step 1). It tells Firebase which project is yours.

Is apiKey a secret? No. It only names your project. Your data is protected by login + rules (step 6).

index.html

```html
<script type="module" src="app.js"></script>
```

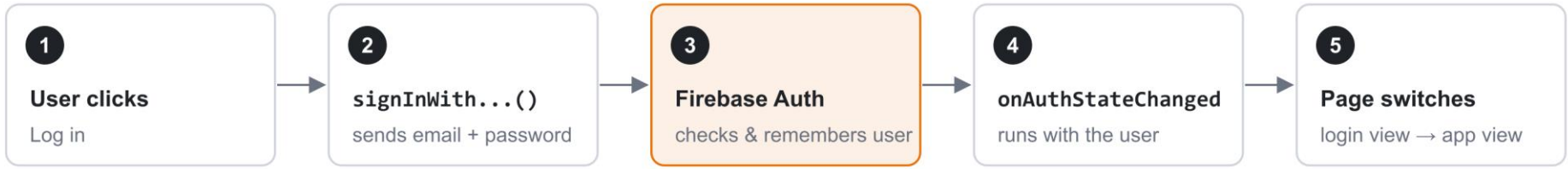
app.js (top)

```js
// 1. Import Firebase from the CDN (no install, no build step)
import { initializeApp } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-app.js";
import {
  getAuth, onAuthStateChanged,
  createUserWithEmailAndPassword, signInWithEmailAndPassword, signOut,
} from "https://www.gstatic.com/firebasejs/10.12.2/firebase-auth.js";
import {
  getFirestore, collection, doc, addDoc, deleteDoc, query, where, onSnapshot,
} from "https://www.gstatic.com/firebasejs/10.12.2/firebase-firestore.js";

// 2. Connect to your project (copy this from the Firebase console)
const app = initializeApp({
  apiKey: "YOUR_API_KEY",
  authDomain: "your-app.firebaseapp.com",
  projectId: "your-app",
  appId: "YOUR_APP_ID",
});
const auth = getAuth(app);
const db = getFirestore(app);
```

STEP 5 · AUTHENTICATION

Log users in with three Firebase functions



try / catch

shows Firebase's error (wrong password, etc.) on the page.

app.js · buttons

```
// 4. Sign up, log in, log out
document.getElementById("signup").addEventListener("click", async () => {
  try {
    await createUserWithEmailAndPassword(auth, email.value, password.value);
  } catch (e) {
    error.textContent = e.message;
  }
});

document.getElementById("login").addEventListener("click", async () => {
  try {
    await signInWithEmailAndPassword(auth, email.value, password.value);
  } catch (e) {
    error.textContent = e.message;
  }
});

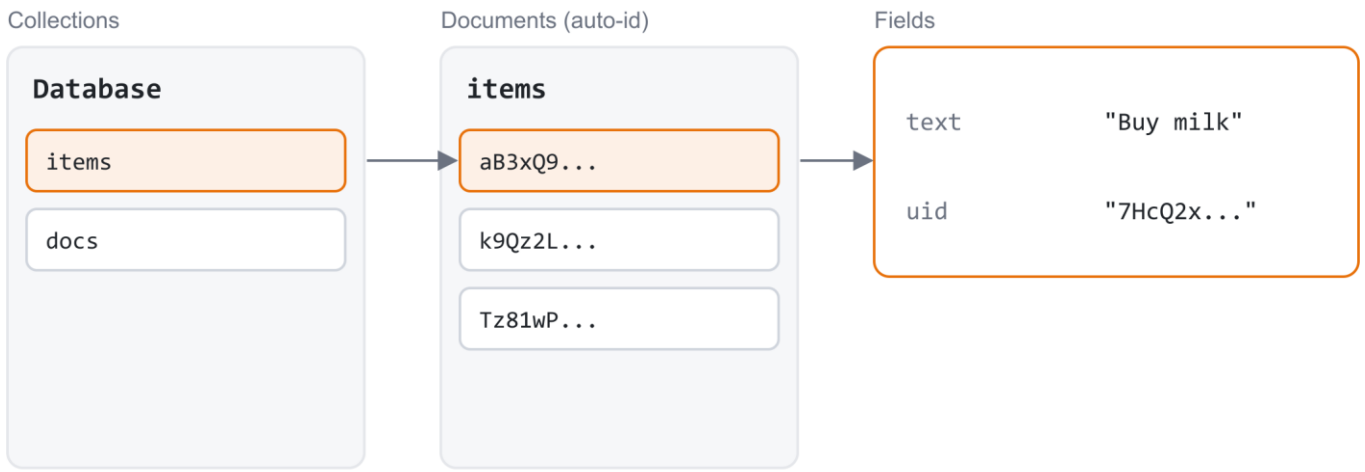
logoutBtn.addEventListener("click", () => signOut(auth));
```

app.js · react to login / logout

```
// 5. Show the right view when the user logs in or out
let stopListening = null;

onAuthStateChanged(auth, (user) => {
  if (user) {
    loginView.hidden = true;
    appView.hidden = false;
    logoutBtn.hidden = false;
    showItems(user);
  } else {
    loginView.hidden = false;
    appView.hidden = true;
    logoutBtn.hidden = true;
    if (stopListening) stopListening();
  }
});
```

Firestore saves data as documents in collections



TO…	USE
Save a new item	`addDoc()`
Read items, live	`onSnapshot()`
Change an item	`updateDoc()`
Delete an item	`deleteDoc()`

Like folders and files

A collection ("items") holds documents. A document holds fields, like a JS object.

Store the user's uid

Saving uid on each item lets you load only that user's items, and lock them with rules.

onSnapshot calls your function again every time the data changes, so the list on screen is always up to date.

STEP 5 · FIRESTORE

Save, show and delete items

app.js · save

```
// 6. Firestore: add, show (live) and delete items
document.getElementById("add").addEventListener("click", async () => {
  if (newItem.value === "") return;
  await addDoc(collection(db, "items"), {
    text: newItem.value,
    uid: auth.currentUser.uid,
  });
  newItem.value = "";
});
```

1 **collection(db, "items")**
Which collection to use. Created automatically on first save.

2 **query + where**
Only this user's items.

3 **onSnapshot**
Clear the list, add one <li> per item. textContent keeps it plain, safe text.

app.js · show + delete

```
function showItems(user) {
  const myItems = query(
    collection(db, "items"),
    where("uid", "==", user.uid)
  );

  // onSnapshot runs now AND every time the data changes
  stopListening = onSnapshot(myItems, (snapshot) => {
    list.innerHTML = "";
    snapshot.forEach((item) => {
      const li = document.createElement("li");
      li.textContent = item.data().text;

      const del = document.createElement("button");
      del.textContent = "Delete";
      del.addEventListener("click", () => {
        deleteDoc(doc(db, "items", item.id));
      });

      li.append(del);
      list.append(li);
    });
  });
}
```

STEP 6 · SHIP

Protect the data, then put it online

firestore.rules (paste in console → Firestore → Rules)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Each item belongs to the user whose uid is stored on it
    match /items/{id} {
      allow read, update, delete: if request.auth.uid == resource.data.uid;
      allow create:               if request.auth.uid == request.resource.data.uid;
    }

  }
}
```

In plain words: you can only see or change an item if its uid is yours.



Rules run on Google's servers. Anyone can edit JavaScript in their browser, but nobody can get past the rules.

terminal

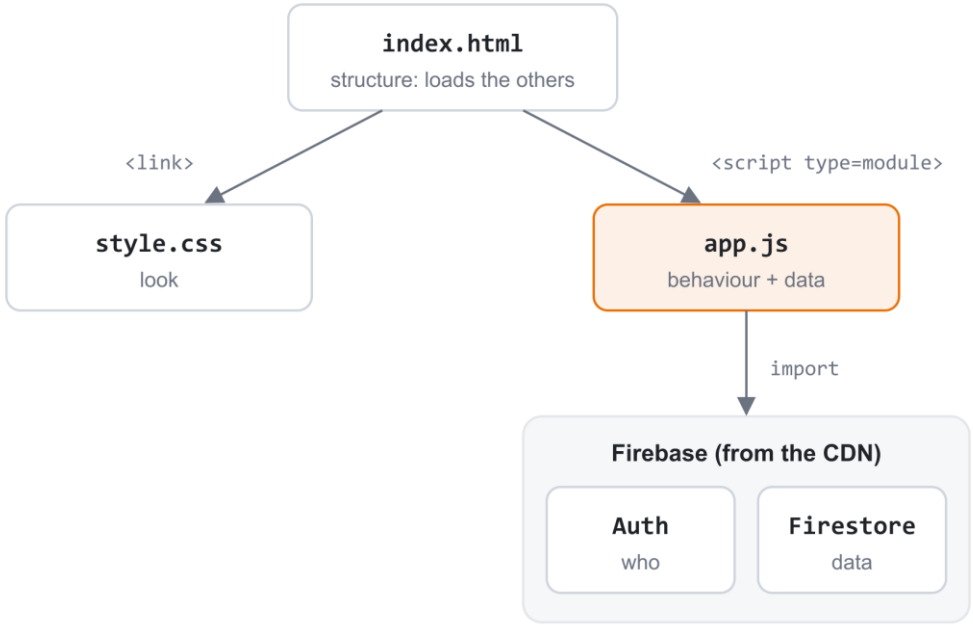
```
npm install -g firebase-tools
firebase login
firebase init hosting   # public folder: .
firebase deploy
```

Your app is now live at your-app.web.app.

Other hosts work too (e.g. Netlify Drop). Add their domain in Authentication → Settings → Authorized domains.

THE REUSABLE TEMPLATE

Three files you can turn into any app



The template is everything from steps 2–6: a login screen and a live list saved in Firestore.

To make a new app from it

1. **Copy the folder**
   Paste your own config into app.js (section 2)
2. **Change the HTML**
   Swap the inputs in #app-view for the ones your app needs
3. **Change the data**
   In section 6, rename "items" and the fields you save and show
4. **Update the rules**
   Use the same collection name
5. **Deploy**
   firebase deploy

index.html: a login view and an app view

index.html (1/2)

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My App</title>
  <link rel="stylesheet" href="style.css">
  <script type="module" src="app.js"></script>
</head>
<body>
  <header>
    <h1>My App</h1>
    <button id="logout" hidden>Log out</button>
  </header>
```

index.html (2/2)

```
  <!-- Shown when logged out -->
  <section id="login-view">
    <input id="email" type="email" placeholder="Email">
    <input id="password" type="password" placeholder="Password">
    <button id="login">Log in</button>
    <button id="signup">Sign up</button>
    <p id="error"></p>
  </section>

  <!-- Shown when logged in -->
  <section id="app-view" hidden>
    <input id="new-item" placeholder="New item">
    <button id="add">Add</button>
    <ul id="list"></ul>
  </section>
</body>
</html>
```

Two views, one page

JavaScript shows one section and hides the other with the hidden attribute: no page reloads.

Every button has an id

That's how app.js finds it: document.getElementById("login").

style.css, and how app.js is organised

style.css

```
body {
  font-family: system-ui, sans-serif;
  color: #1f2328;
  max-width: 640px;
  margin: 40px auto;
  padding: 0 16px;
}

header { display: flex; justify-content: space-between; align-items: center; }

input  { padding: 8px; border: 1px solid #ddd; border-radius: 6px; }

button {
  padding: 8px 14px;
  border: none;
  border-radius: 6px;
  background: #e8710a;
  color: white;
  cursor: pointer;
}

ul     { padding: 0; }
li     { display: flex; justify-content: space-between; padding: 10px 0;
         border-bottom: 1px solid #eee; list-style: none; }
#error { color: crimson; }
```

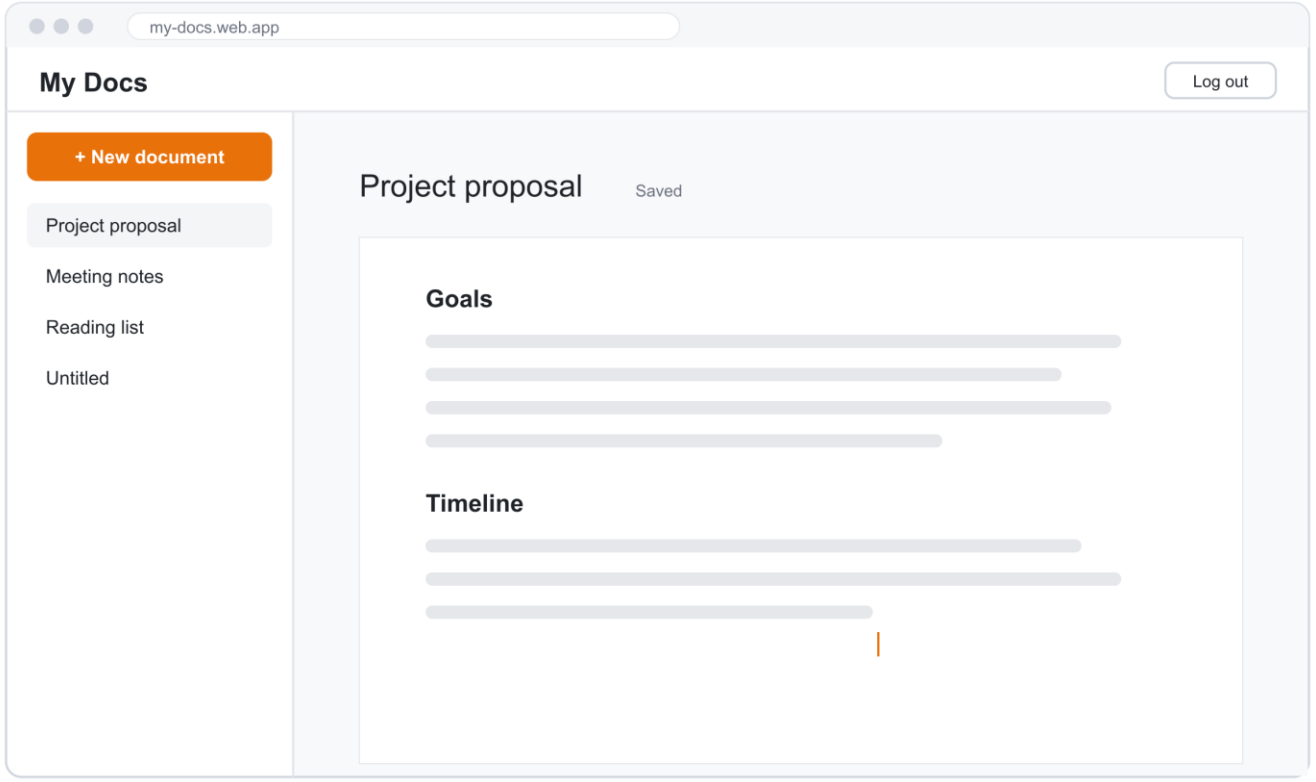
app.js in six sections

1	**Import Firebase** from the CDN	same in every app
2	**Connect** your config	same
3	**Grab elements** getElementById	**your ids**
4	**Sign up / in / out** button clicks	same
5	**Switch views** onAuthStateChanged	same
6	**Your data** add, show, delete	**your app**

Orange steps are the only parts you change.

EXAMPLE · DOCS APP

A minimal Google Docs, built from the template



- Sidebar: a live list of your documents
- A white "page" you type on (contenteditable)
- Autosave 1 second after you stop typing
- Ctrl+B and Ctrl+I already work in the page

```
one Firestore document

docs/{id}
  title  "Project proposal"
  body   "<b>Goals</b><div>...</div>"
  uid    "7HcQ2x..."
```

Sections 1–5 of app.js are copied unchanged. Only the HTML, the CSS and section 6 are new.

Rules: the same as the template, with "docs" instead of "items".

EXAMPLE · DOCS APP

Layout: a sidebar and a paper page

index.html · the new #app-view

```
<section id="app-view" hidden>
  <aside>
    <button id="new-doc">+ New document</button>
    <ul id="doc-list"></ul>
  </aside>
  <main>
    <input id="title" placeholder="Untitled">
    <small id="status"></small>
    <div id="page" contenteditable="true"></div>
  </main>
</section>
```

contenteditable="true"

turns a plain into a text editor. No library needed.

Why the [hidden] rule?

display: grid would override the hidden attribute, so we hide it again explicitly.

style.css · added rules

```
/* Two columns: sidebar + editor */
#app-view { display: grid; grid-template-columns: 220px 1fr; gap: 32px; }
#app-view[hidden] { display: none; }

#doc-list    { padding: 0; }
#doc-list li { list-style: none; padding: 8px; cursor: pointer; }
#doc-list li:hover { background: #eceef1; }

#title  { font-size: 24px; border: none; background: none; width: 70%; }
#status { color: #6b7280; }

/* The "paper" */
#page {
  min-height: 900px;
  margin-top: 16px;
  padding: 64px;
  background: white;
  box-shadow: 0 1px 4px rgba(0, 0, 0, 0.15);
  outline: none;
}
```

EXAMPLE · DOCS APP

Create, open and autosave documents

app.js · section 6

```
// 6. Documents
let currentId = null;   // id of the open document
let timer = null;

// Create a new, empty document and open it
document.getElementById("new-doc").addEventListener("click", async () => {
  const newDoc = await addDoc(collection(db, "docs"), {
    title: "Untitled",
    body: "",
    uid: auth.currentUser.uid,
  });
  openDoc(newDoc.id, "Untitled", "");
});

// Put a document into the editor
function openDoc(id, title, body) {
  currentId = id;
  titleInput.value = title;
  page.innerHTML = body;
}
```

updateDoc() changes the fields you pass and leaves the rest alone.

app.js · autosave

```
// Autosave: wait until typing stops for 1 second, then save
function save() {
  if (currentId === null) return;
  statusText.textContent = "Saving...";
  clearTimeout(timer);
  timer = setTimeout(async () => {
    await updateDoc(doc(db, "docs", currentId), {
      title: titleInput.value,
      body: page.innerHTML,
    });
    statusText.textContent = "Saved";
  }, 1000);
}
titleInput.addEventListener("input", save);
page.addEventListener("input", save);
```

How autosave works

Each keystroke cancels the last timer and starts a new 1-second one, so it saves once you pause, not on every key.

The sidebar

showDocs() is the template's showItems() with "docs": each <li> opens its document when clicked.

RECAP

Same recipe, any app

1. **HTML**
   Elements with ids
2. **CSS**
   Selectors + properties
3. **DOM**
   Find, listen, create, update
4. **Firebase**
   Import from CDN, Auth + Firestore
5. **Ship**
   Rules + firebase deploy

Ideas for what to add next

Google sign-in
signInWithPopup(auth, new GoogleAuthProvider())

Sort by date
Save a date field and sort the list in JavaScript

Share a doc
Save a list of emails on it and allow them in the rules

Code: the template and the docs app are in the template/ and docs-app/ folders next to this deck.